

金絲雀部署的可觀測性實踐

APISIX + OPA + OTeI 打造安全佈署

Duran Hsieh | GitHub Star · Microsoft MVP · TSMC Section Manager



APISIX

OPA

OpenTelemetry

Grafana

ArgoCD

你遇過這些問題嗎？

>> 週四上午部署了新版本，全公司使用者同時踩到 bug

沒有金絲雀保護

>> 工程師凌晨三點被叫起來，切回穩定版操作錯誤，事故擴大

沒有快速回滾機制

>> 明明有金絲雀佈署，事故發生時沒人知道 canary 錯誤率已高了 5 倍

沒有可觀測性

[!] 佈署不是問題，看不見才是問題

今天的三個核心命題

01

如何讓「對的人」先測試新版本？

→ OPA 策略驅動路由

02

如何在事故發生前就「看到」問題？

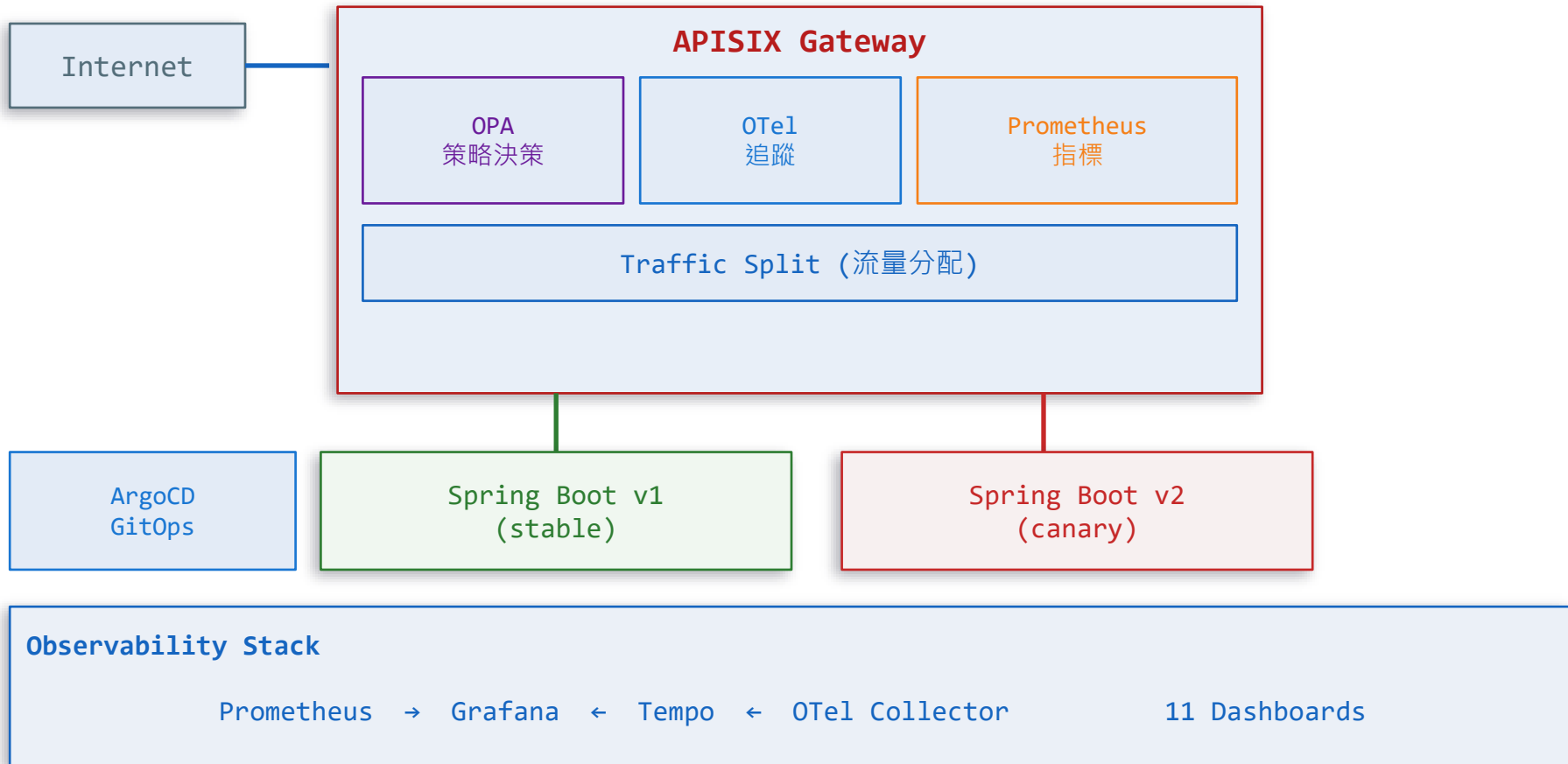
→ 可觀測性：Metrics + Traces + Logs

03

如何用數據驅動佈署決策，而非靠直覺？

→ 漸進式金絲雀 + SLO Metrics

架構總覽



為什麼選 **APISIX** ?

[+] 高效能

基於 Nginx/OpenResty
高併發低延遲

[~] 原生插件支援

traffic-split / prometheus
opentelemetry / opa

[o] 動態配置

Admin API 即時調整權重，不需重啟
調整後立即推送至 Git Repository，留下紀錄

[*] 豐富生態

80+ 個插件
安全 · 限流 · 認證 · 可觀測

APISIX vs Nginx Ingress

功能	APISIX	Nginx Ingress
金絲雀切換	Admin API 一鍵切換	annotation + re-apply
OPA 整合	原生插件	需要額外 sidecar
動態配置	[v] 不需重啟	[x] 需要 reload

OPA 在金絲雀佈署中的角色

```
# canary.rego - 核心邏輯

package apisix.canary

# Header-based: QA 帶 x-canary
canary_by_header if {
    input.request.headers["x-canary"] == "true"
}

# 路由決策
route_target := "canary" if { canary_by_header }
route_target := "stable" if { not canary_by_header }
```

使用場景

QA 測試

x-canary: true → canary

內部員工

canary=true cookie → canary

微服務鏈路

服務間附帶 header → 整鏈路 canary

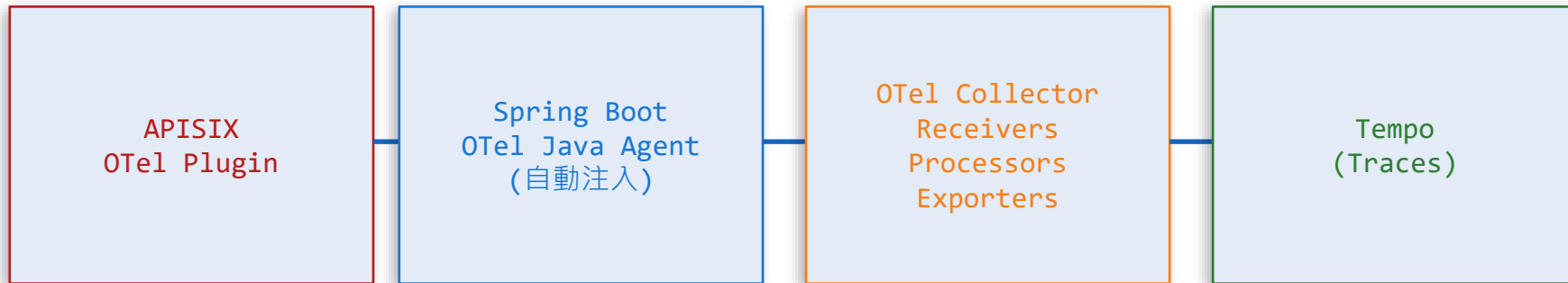
一般使用者

無 header/cookie → stable

OPA Bundle 更新流程

Rego 更新 → CI/CD 打包 bundle.tar.gz → Azure Blob Storage → OPA 每 30-60 秒自動拉取 → 策略即時生效

OpenTelemetry 在架構中的定位



關鍵設計 — service.name 區分版本

Stable | demo-api-stable | env:
production

Canary | demo-api-canary | env:
canary

Zero Intrusion — 不需要改一行 Java 程式碼

```
ENV JAVA_TOOL_OPTIONS="-javaagent:/otel-agent.jar" OTEL_EXPORTER_OTLP_ENDPOINT="http://otel-collector:4318"
```

雙版本並行：Stable / Canary 架構

Namespace: app

Deployment: spring-boot-stable (replicas: 2)

APP_VERSION=v1
OTEL_SERVICE_NAME=demo-api-stable
Service: spring-boot-stable:8080

Deployment: spring-boot-canary (replicas: 2)

APP_VERSION=v2
OTEL_SERVICE_NAME=demo-api-canary
Service: spring-boot-canary:8080

流量控制雙層架構

Layer 1 – OPA 精確路由

x-canary: true → canary
canary cookie → canary
預設 → stable

Layer 2 – APISIX 權重路由

stable: weight 100 → 100% 流量
canary: weight 0 → 0% 流量
(透過 Admin API 動態調整，即時生效)

```
curl http://$LB_IP/api/hello → {"version":"v1"} | curl -H "x-canary: true" ... → {"version":"v2"}
```


11 個 Grafana Dashboards — 全方位監控

APISIX Overview

總覽與健康

APISIX Route Metrics

路由與流量

Canary Traffic Split

* 金絲雀核心

Spring Boot Metrics

應用層

Node Metrics

基礎設施

Upstream Health

後端服務

Security & Rate Limiting

安全防護

SLI / SLO

穩定服務

Plugin Metrics

插件與擴展

Nginx/OpenResty

底層系統

Performance & Cache

效能與快取

11

Dashboards
60+ 監控面板

Canary Traffic Split Dashboard

金絲雀核心儀表板 — 這是你的佈署指揮中心

流量分配 (Pie Chart)

Stable vs Canary
即時百分比

QPS 對比

stable / canary
雙軸折線圖

P95 延遲對比

stable / canary
閾值標線

5xx 錯誤率

canary 5xx
> stable × 5?

Tempo Traces

Trace by
service.name

Traffic Anomaly

流量突增/
歸零告警

[!] Dashboard 不只是「看的」— 每個面板都要有對應的操作 SOP

APISIX 核心監控指標

請求指標

`apisix_http_status`

`apisix_http_latency_bucket` (P50/P95/P99)

`type: request / upstream / apisix`

連線指標

`apisix_nginx_http_current_connections`

`state: active / waiting / reading / writing`

健康指標

`apisix_etcd_reachable` (1=UP, 0=DOWN)

`apisix_upstream_status`

`apisix_etcd_modify_indexes`

記憶體 / 資源

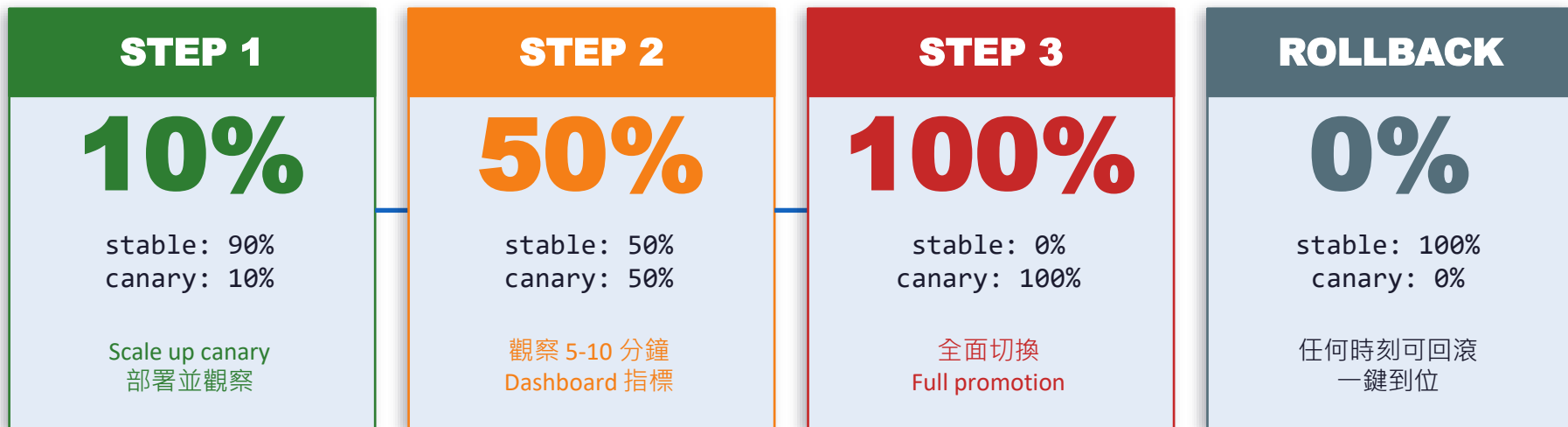
`apisix_shared_dict_capacity_bytes`

`apisix_shared_dict_free_space_bytes`

`apisix_node_info`

ServiceMonitor scrape interval: 15s | metric_prefix: apisix_ | port: 9091

漸進式金絲雀擴散策略



```
./scripts/canary-switch.sh --stable 90 --canary 10 # 即時生效，無需 redeploy
```

[Q] 先看再切

|| 異常就暫停

<- 錯誤就回滾

[t] 每階段觀察 10 分鐘

決策指標：什麼時候該停？什麼時候該進？

指標	 綠燈 — 安全繼續	 黃燈 — 謹慎觀察	 紅燈 — 立即回滾
P95 延遲	< 200ms	200ms ~ 1000ms	> 1000ms
5xx 錯誤率	< 1%	1% ~ 5%	> 5%
Canary vs Stable 延遲差	< 20%	20% ~ 50%	> 50%
OPA 策略錯誤	0	> 0	持續增加

對應 Prometheus Alert Rules

```
- alert: CanaryHighErrorRate
  expr: canary 5xx > 5% | for: 2m | severity: critical
  annotations.runbook: "./scripts/canary-switch.sh --stable 100 --canary 0"

- alert:
  CanaryHighLatency (P95 > 1s, for: 3m)
```

Alert 的角色 — 告警即行動指令

CanaryHighErrorRate	canary 5xx > 5%	for: 2m	critical
CanaryHighLatency	canary P95 > 1s	for: 3m	critical
CanaryLatencyDrift	canary 延遲 > stable × 1.5	for: 5m	warning
CanaryTrafficAnomaly	流量突然歸零或暴增	for: 1m	warning
UpstreamUnhealthy	健康檢查失敗	for: 1m	critical

告警要 Actionable

- >> 什麼壞了？ → Canary 延遲飆升
- >> 影響是什麼？ → 10% 使用者體驗下降
- >> 該怎麼做？ → 執行 `canary-switch.sh`

告警管道

- On-call
- 通訊軟體通知
- Webhook 自動回滾

可觀測性三支柱在本架構的對應

[M] Metrics

工具

Prometheus + Grafana

資料來源

APISIX prometheus plugin
Spring Boot Actuator
OTel Collector

用途

QPS · 延遲 · 錯誤率 · JVM 狀態

[Q] Traces

工具

OTel + Tempo + Grafana

資料來源

APISIX OTel plugin
Spring Boot OTel Java Agent

用途

請求鏈路追蹤
找出版本間的行為差異

[L] Logs

工具

EFK/ELK + Kibana

資料來源

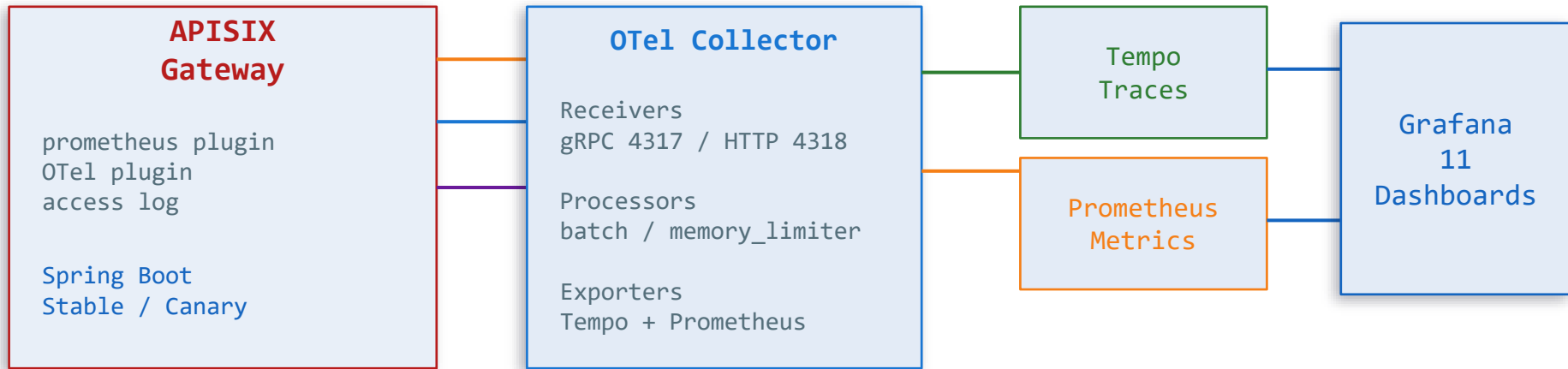
Spring Boot stdout
APISIX access log

用途

錯誤詳情 · 除錯分析
Trace-ID 關聯

Kibana：按 x-route-to header 區分 stable/canary 日誌 | 按 trace-id 關聯完整鏈路

資料流全景圖



Prometheus AlertManager → OnDuty | 通訊軟體 | 自動回滾腳本

Log Pipeline: Filebeat/Fluentd → Elasticsearch → Kibana (x-route-to header 分版本)

ArgoCD GitOps ← watches this repo | 所有配置版本化 · 可審計可回溯

金絲雀佈署 + 可觀測性：8 大最佳實踐

01

GitOps First

所有配置版本化，ArgoCD 自動同步，可審計可回溯

02

Policy as Code

OPA 策略走 CI/CD，有 review、有測試、有紀錄

03

漸進式佈署

永遠不要 0→100%，最少三階段 (10% → 50% → 100%)

04

SLO-driven Rollout

用 SLI/SLO 指標 (延遲、錯誤率) 決定是否繼續佈署

05

自動回滾

Prometheus Alert + Webhook，錯誤率超標自動觸發回滾

06

Trace-based Testing

Canary 階段利用 distributed tracing 比對新舊版本行為

07

Dashboard as Runbook

Dashboard 附帶 SOP：指標異常 → 該做什麼操作

08

Chaos Engineering

定期注入故障，驗證可觀測性和告警是否正常運作

與業界方案比較

方案	特色	可觀測性整合	複雜度
Argo Rollouts	K8s 原生，自動 progressive delivery	Prometheus Analysis	中
Flagger	搭配 Istio/Linkerd，自動金絲雀	內建 metric check	中高
Istio + Kiali	Service Mesh 完整方案	完整但複雜度高	高
* 本架構	策略驅動、閘道層控制，無 Service Mesh	Metrics + Traces + Logs	中低

本架構的差異化優勢

[v] 不需要 Service Mesh → 降低複雜度與資源消耗

[v] 閘道層觀測 + 應用層觀測 → 雙管齊下

[v] 11 個 Dashboard → 企業級全方位監控

[v] OPA 提供比 annotation 更靈活的策略控制

[v] Admin API 動態切換 → 不需重新部署

[v] GitOps → 所有操作可追溯、可重現

Key Takeaways

1

[v] 金絲雀佈署不難，難的是看見它

沒有可觀測性的金絲雀 = 盲人開車

3

[v] 漸進式擴散 + SLO 閾值 = 數據驅動佈署決策

讓指標說話，不靠直覺

5

[v] Dashboard 是你的佈署 Runbook

每個面板都要有對應的操作 SOP

2

[v] OPA 讓你精準控制誰先走新路

不再是「全部使用者一起賭」

4

[v] Metrics + Traces + Logs 三支柱缺一不可

Prometheus / Tempo / Kibana 各司其職

6

[v] GitOps 確保可重現性

所有操作都有紀錄，出事能追溯

感謝聆聽！

TSMC IT TSID 徵才



如果你對下列工作有興趣，歡迎找我聊聊

- Site Reliability Engineer
- DevOps Engineer (#Backend #Java)

參考資料

APISIX
apisix.apache.org

OPA
openpolicyagent.org

OpenTelemetry
opentelemetry.io

Grafana Tempo
grafana.com/oss/tempo

Questions?